quickly.

Best practices include:

- Use hashed passwords via bcrypt or Argon2id.
- Enforce strong password rules.
- Regenerate session IDs upon login to prevent session fixation.
- Use policies and gates for fine-grained access control.
- Limit login attempts using `ThrottleRequests` middleware.

16.7 Securing Sessions and Cookies

Laravel sessions are encrypted and stored securely by default. To enhance this: - Set `SESSION_SECURE_COOKIE=true` to transmit cookies only over HTTPS. - Use `httpOnly` and `secure` flags in cookie settings. - Store sessions in a database or Redis for better control and scalability.

16.8 Force HTTPS and TLS

Using HTTPS is essential to protect data in transit. Laravel allows you to enforce HTTPS:

- Add `\App\Http\Middleware\EnsureHttps::class` middleware.
- Set `APP_URL=https://yourdomain.com` in `.env`.
- Redirect HTTP traffic to HTTPS via `.htaccess` or web server config.

16.9 Secure File Uploads

File uploads are often a major attack surface. Laravel allows you to handle file uploads securely:

- Always validate file types and sizes.
- Rename uploaded files to avoid guessing.
- Store sensitive files outside `public/` directory.
- Scan uploaded files if they will be executed or viewed.